

Array-Native Randomness: An Efficient Unified Permutation-Generation Architecture with Deterministic and High Sensitivity Properties

1st Rahman

Informatics Engineering)
State Polytechnic of Malang
Malang, Indonesia
nextthamas95@gmail.com

Abstract—In modern data-intensive systems, such as large-scale data processing and AI/ML pipelines, efficient and reproducible data transformation is as critical as high-quality random number generation. Conventional approaches, such as the Fisher–Yates shuffle, rely on repeated scalar RNG calls, introducing overhead and limiting scalability in high-throughput environments. [1]

This paper introduces `ra_prng2`, a deterministic array-based pseudo-random number generation architecture that unifies random generation and permutation into a single in-place process. Unlike traditional scalar or counter-based PRNGs, `ra_prng2` operates on two internal (32×256) arrays that simultaneously serve as state and entropy reservoir. Through continuous mutation using bitwise rotation, XOR, shifting, and nonlinear indexing, the proposed design enables efficient generation while inherently performing structured data shuffling.

The relatively small state size (2 KB) fits within typical L1 cache capacities, which may contribute to the observed high instruction-level efficiency, with IPC consistently above 3. Extensive statistical evaluation demonstrates that `ra_prng2` passes NIST STS, Dieharder, PractRand, and BigCrush test suites, while maintaining near-ideal entropy distribution and strong avalanche behavior (49.9% bit-flip sensitivity). The estimated theoretical period reaches approximately $2^{4969.65}$, with further extensions in `ra_prng3`.

Beyond conventional random generation, we observe that the deterministic state evolution exhibits high sensitivity to both initial seeds and optional input data, where minor perturbations lead to significant divergence in output states. This property suggests potential applications in lightweight data fingerprinting and reproducible data transformation pipelines.

The proposed architecture offers a unified, deterministic, and high-efficiency approach to randomness and permutation, making it suitable as a system-level primitive for modern scalable computing environments.

Index Terms—pseudorandom number generation, array permutation, deterministic systems, avalanche effect, in-place computation, benchmarking, Fisher-Yates shuffle, entropy, data fingerprinting, Lemire’s fast reduction

I. INTRODUCTION

High-throughput computing systems that depend on randomness — including dataset preprocessing, reproducible simulation, and stochastic transformation pipelines — commonly treat random number generation and data permutation as separate, sequentially dependent processes. The Fisher–Yates

shuffle exemplifies this coupling: each index requires an independent RNG call, binding permutation throughput directly to generator latency. While this overhead is tolerable at small scale, it becomes a structural inefficiency as data volumes grow. Penschuck et al. [2] demonstrated that shared-memory parallel shuffling can redistribute permutation costs across workers, while showing that the dominant performance constraints increasingly arise from memory access behavior rather than from the shuffling logic itself. This shift suggests that further gains may require revisiting adjacent components of the pipeline, including the architecture of the random generator that feeds permutation-intensive workloads.

Conventional PRNGs, including PCG, Xoshiro, and Mersenne Twister, are scalar systems in which a recurrence produces values that subsequently drive external transformations. In this model, randomness and transformation are structurally decoupled. The generator is unaware of the data being shuffled yet the shuffler is unaware of the generator’s internal state. This separation is not an implementation choice but a foundational assumption of scalar PRNG design. As a result, to our knowledge there is no existing general-purpose PRNG architecture natively integrates permutation as part of its state evolution — shuffling remains an application of randomness, never its source.

This observation suggests a deeper question: instead of using random numbers to drive permutation, can permutation itself act as the generative process? Specifically, is it possible to design a deterministic system where state evolution through structured permutation inherently produces pseudo-random behavior, eliminating the need for externally driven index generation? We introduce `ra_prng2`, an array-native architecture in which the internal state consists of two 256-element arrays that undergo continuous in-place permutation and nonlinear mutation. Randomness in `ra_prng2` is not an input to shuffling — it is an emergent property of the permutation process itself. This structural inversion eliminates the call boundary between generator and shuffler, allowing both processes to proceed within a single execution loop without external function calls or reseeding overhead.

This paper makes three concrete contributions. First, we

present the `ra_prng2` architecture and its formal state transition model, demonstrating how unified permutation-generation eliminates RNG call overhead in transformation-heavy workloads. Second, we provide comprehensive benchmarking against established PRNGs — including xoshiro256, PCG32, Philox4x32, and ChaCha20 — across throughput, shuffling performance, IPC, and statistical quality, establishing `ra_prng2` as a measurable reference point for array-based generator design. Third, we identify and characterize a sensitivity property of the architecture: minor perturbations to the initial seed produce immediate and sustained divergence in output state across three independent perturbation models, suggesting `ra_prng2` as a candidate primitive for lightweight data fingerprinting in deterministic transformation pipelines.

The remainder of this paper is organized as follows. Section II describes the architecture and formal design of `ra_prng2`, including its state transition operator, initialization procedure, and data-shuffling integration. Section III presents the performance evaluation, covering statistical quality, avalanche analysis, throughput benchmarking, and periodicity estimation. Section IV discusses implications for fingerprinting applications and directions for future work. Section V concludes.

II. STRUCTURE OF RA_PRNG2 ALGORITHM

The structure of `ra_prng2` is built from a large internal state consisting of 2 arrays containing 256 32-bit numbers each, which evolves through repeated permutation and nonlinear bitwise transformations. At each outer step, one array is iteratively permuted while outputs are extracted from nonlinear state interactions. The second array is then mixed with the transformed state and used to derive the next control scalar.

$$\mathbb{S} = \mathbb{Z}_{2^{32}}^{256} \times \mathbb{Z}_{2^{32}}^{256} \times \mathbb{Z}_{2^{32}} \times \mathbb{Z}_{2^{32}}$$

where we denote an internal state at discrete step t as

$$\begin{aligned} S_t &= (L_t, M_t, it_t, cons_t), \\ L_t, M_t &\in \mathbb{Z}_{2^{32}}^{256}, \\ it_t, cons_t &\in \mathbb{Z}_{2^{32}}. \end{aligned}$$

The algorithm defines a deterministic state transition operator

$$F : \mathbb{S} \rightarrow \mathbb{S}, \quad S_{t+1} = F(S_t),$$

which can be decomposed conceptually as a composition of primitive operators

$$F = \text{Reseed} \circ \text{Mutate} \circ \text{Permute},$$

where `Mutate` denotes bitwise nonlinear transformations on array elements, `Permute` denotes index permutations (swaps) on L , and `Reseed` computes the next scalar $cons$ via a hash-like function on M .

$$\text{rotl}_{32}(x, r) : ((x \ll r) \vee (x \gg (32 - r))) \bmod 2^{32},$$

and we denote bitwise XOR by $\oplus$, left shift by $\ll$ and right shift by $\gg$.

For an outer iteration t , and for each inner index $i \in \{255, \dots, 0\}$, the algorithm computes intermediate variables a, b, c, d, o . For each inner iteration, a is initialized from the current scalar $cons$ (stable entropy) and b from the outer loop index it (volatile entropy), both evolve through the inner loop transformations., o is an aggregated mixer derived from M , and d is the bounded swap index. The value of o from M is formalized as

$$\begin{aligned} e &:= 0, \dots, 7, \\ o_{(0)} &:= 0, \\ o_{(e+1)} &:= o_{(e)} \oplus (M_t[(i + e) \bmod 256] \ll e), \\ o &:= o_{(8)} \end{aligned}$$

The core transforms per index i are then:

$$\begin{aligned} a' &= \text{rotl}_{32}(b \oplus o, d) \oplus (cons_t + a), \\ b' &= \text{rotl}_{32}(cons_t + a', i) \oplus (o + d), \\ o' &= (\text{rotl}_{32}(a' \oplus o, i) \ll 9) \oplus (b' \gg 18), \end{aligned}$$

and the output register update

$$c' = \text{rotl}_{32}((o' + (c \ll 14)) \oplus (b' \gg 13) \oplus a', b').$$

The produced output for this inner step is

$$O_{t,i} = c'.$$

The random index d is computed using Lemire's fast reduction; for 64-bit intermediate multiplication we write:

$$d = \left\lfloor \frac{((O_{t,i} \bmod 2^{64}) \times (i + 1)) \bmod 2^{64}}{2^{32}} \right\rfloor$$

which in implementation reduces to the commonly used 64-bit to 32-bit reduction as described in [3].

Swapping the array entries at indices i and d implements the permutation:

$$L'_t = \text{swap}(L_t, i, d), \quad \text{i.e.} \quad L'_t[i] = L_t[d], \quad L'_t[d] = L_t[i],$$

and all other indices unchanged. This defines one elementary permutation applied inside `Permute`.

After the inner loop completes (for all i), the mutation of M is an elementwise XOR with the permuted L :

$$M_{t+1}[j] = M_t[j] \oplus L'_t[j], \quad j = 0, \dots, 255.$$

The reseed scalar is computed by a hash-like reduction $\text{ra_hash} : \mathbb{Z}_{2^{32}}^{256} \rightarrow \mathbb{Z}_{2^{32}}^8$,

$$\text{tmp8} = \text{ra_hash}(M_{t+1}), \quad \text{new_cons} = \bigoplus_{e=0}^7 (\text{tmp8}[e] \ll e),$$

and the next scalar state is

$$cons_{t+1} = \text{new_cons}.$$

Thus one full outer transition can be summarized compactly by

$$S_{t+1} = F(S_t) = (L_{t+1}, M_{t+1}, it_t, cons_{t+1}),$$

and the observable output stream is the sequence $\{O_{t,i}\}$ over iterations t and inner indices i . Here, it denotes a loop-local iteration variable reset to zero at each outer step; it is included in the state description for completeness but does not persist across outer transitions.

1) *Array State and Seed Initialization*: random generation begins by constructing two 32-bit arrays, $L[256]$ and $M[256]$. Both are filled with specific constants based on predetermined values. By default, the arrays are initialized as shown below. In addition, the seed is initialized at the same time; however, in fact, seed initialization is optional because the entropy weight derived from the two lists is sufficient to create random data and the seed in the next iterations. In other words, the seed value can be between 0 to $2^{32} - 1$ without affecting the output quality.

Algorithm 1. Seed and array initiation phase.

```

Initialize: seed  $\leftarrow$  any positive integer (1 in default)
1 for  $i \leftarrow 0$  to 255 do
2    $L[i] \leftarrow i \times 0x06a0dd9b + 0x06a0dd9b$ ;
3    $M[i] \leftarrow i \times 0x9e3779b7 + 0x9e3779b7$ ;
4    $*[r]$ Initiating 256 random item per array

```

These constants are presented as fixed multipliers, derived from the golden ratio value, which is also used by other PRNGs such as PCG or Xoshiro256, to reduce collisions, dead states, and short cycles by applying the following transformations repeatedly.

2) *Core Algorithm: the ra_core Loop*: The execution structure of ra_prng2 is visualized in this ra_prng2 flowchart, showing the crucial flow from the beginning to the very end of the algorithm.

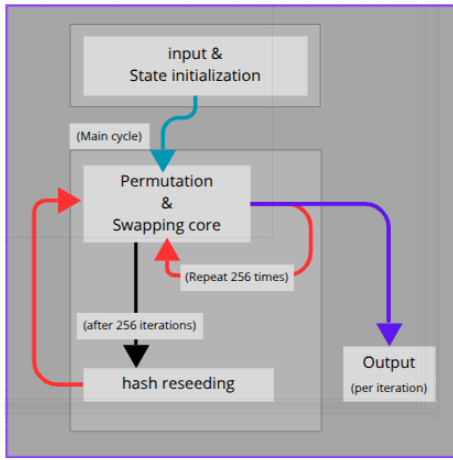


Fig. 1. Flow of ra_prng2 algorithm.

The main function of ra_core executes a number of iterations that act as internal state randomizers, provide output, and supply entropy for the next iterations, as well as generate random data. Each iteration changes the shape of arrays L and M through a combination of: bitwise rotation (rot32), XOR operation, bit shifting, and index swapping between $L[i]$

and $L[d]$, based on the index that has been calculated from the random data created inside the loop. Followed by XOR between all items in arrays L and M as entropy diffusion, and reseeding ($cons$) through a simple XOR and bit shift-based hash function.

Algorithm 2. Main function of ra_prng2

```

Initialize:  $cons \leftarrow$  seed;  $it \leftarrow 0$ ; iterations  $\leftarrow$  any positive integer
1 for  $it \leftarrow 0$  to iterations-1 do
   // Initialize variables
2    $a \leftarrow cons$ ; // stable entropy
3    $b \leftarrow it$ ; // volatile entropy
4    $c \leftarrow 0$ ; // output accumulator
5    $d \leftarrow 0$ ; // random index buffer
6   for  $i \leftarrow 255$  downto 0 do
7      $o \leftarrow 0$ ; // main entropy source
8     for  $e \leftarrow 0$  to 7 do
9        $o \leftarrow o \oplus (M[(i+e) \bmod 256] \ll e)$ 
10       $a \leftarrow ROTL_{32}(b \oplus o, d) \oplus (cons + a)$ ;
11       $b \leftarrow ROTL_{32}(cons + a, i) \oplus (o + d)$ ;
12       $o \leftarrow (ROTL_{32}(a \oplus o, i) \ll 9) \oplus (b \gg 18)$ ;
   // Generate RNG output
13       $c \leftarrow ROTL_{32}(((o + (c \ll 14)) \oplus (b \gg 13) \oplus a), b)$ ;
   // Get random index using Lemire's fast reduction
14       $d \leftarrow \lfloor ((c \times (i+1)) \bmod 2^{64}) / 2^{32} \rfloor$ ;
15      swap( $L[i], L[d]$ );
   // Mutate array M after full shuffle
16 for  $i \leftarrow 0$  to 255 do
17    $M[i] \leftarrow M[i] \oplus L[i]$ 
   // Regenerate constant cons
18  $tmp8 \leftarrow ra\_hash(M)$ ;
19  $new\_cons \leftarrow 0$ ;
20 for  $e \leftarrow 0$  to 7 do
21    $new\_cons \leftarrow new\_cons \oplus (tmp8[e] \ll e)$ 
22  $cons \leftarrow new\_cons$ ;

```

3) *Technical Purpose & Significance*:

- the value of variable c in each loop, before being used for random indexing of array L through Lemire's Reduction function, is the value taken as the output of random generation. Lemire's fast modulo reduction [3] efficiently indexes n -bit integers to a smaller size without the common statistical bias that arises in costly modulo operations ($n \bmod i$).
- Swapping $L[i]$ with $L[d]$ implements internal non-linear index permutation, making it a PRNG capable of shuffling its own internal state.
- All operations of this algorithm are branchless, speeding up, compacting execution, and optimizing CPU pipeline

A. *Output Production: 32-bit numbers*

In each iteration i , the internal loop runs backward from 255 to 1, and each loop produces one valid value (c) as 32-bit random output. Thus, one iteration can handle up to 256 random generating before reseeding for the next iteration. [4].

B. PRNG Reseeding

After 255 internal loops and array swapping are performed, this means the entire entropy of array M has been used up. Therefore, the next step is to update all values of M by performing XOR against array L which has been fully shuffled before, and reseed the constant value through the `ra_hash` function to maximize state space coverage. The full implementation of the hash function is provided in the appendix [4]. The new constant is constructed by XORing all output values of the hash (`tmp8`) with varying bit shifts for each value, preserving bit sensitivity (avalanche effect).

C. Data-Shuffling Application

In specific applications of data randomization, random indexing can be treated with the same technique as shuffling an array L . The shuffling code is broadly written as follows, for more detailed code, see the GitHub repository.

Algorithm 3. `ra_prng2` implementation in shuffling tool

```
// Generate RNG output
1  $c \leftarrow \text{ROTL}_{32}(((o + (c \ll 14)) \oplus (b \gg 13)) \oplus a), b)$ ;
2  $\text{idx} \leftarrow \lfloor ((c \times (\text{count} + 1)) \bmod 2^{64}) / 2^{32} \rfloor$ ;
   // Get random index for the data
3  $d \leftarrow \lfloor ((c \times (i + 1)) \bmod 2^{64}) / 2^{32} \rfloor$ ;
   // Get random index for internal state
4  $\text{swap}(L[i], L[d])$ ;
5  $\text{swap}(\text{data}[\text{idx}], \text{data}[\text{count}])$ ;
```

III. PERFORMANCE EVALUATION, STATISTICAL ANALYSIS, AND PERIODICITY

A. Testing Methodology

To ensure comparability with prior works, the performance evaluation employed four categories of tests that are commonly adopted in PRNG benchmarking.

- 1) Classic randomness tests using `ent`, NIST STS, and Dieharder.
- 2) Entropy tests and avalanche-effect measurement to assess input-output sensitivity.
- 3) PRNG periodicity analysis.
- 4) Shuffling performance benchmarking, and practical application comparison.

These test suites (NIST STS, Dieharder, `ent`) are widely recognized and commonly adopted in the evaluation of PRNGs.

1) *Entropy, Distribution, Dieharder, and NIST STS Results:* An initial entropy test of `ra_prng2` output using the `ent` tool [5] on a 1 GiB sample yielded an entropy value of 7.99999 bits/byte, nearly the theoretical maximum of 8 bits/byte [6]. The byte distribution showed a mean of 127.5018, very close to the ideal random expectation of 127.5. A Monte Carlo estimation of π produced 3.14133013 with a relative error of 0.01%. The serial correlation coefficient was measured at 0.000001, indicating negligible correlation between samples.

Across all 188 subtests in the NIST Statistical Test Suite (STS) [7], the mean p -value was 0.503 ± 0.287 , consistent with the uniform $[0,1]$ expectation for ideal random sources.

The overall pass rate reached 99.4%, with no subtest below the recommended significance threshold of $p > 0.0001$. This shows stable diffusion of entropy throughout the generator's state.

Meanwhile, in Dieharder [8] testing (total of 114 subtests), `ra_prng2` passed 112 subtests (PASSED) and yielded 2 WEAK results, with zero FAILED outcomes. The WEAK subtests were `rgb_lagged_sum 24` (p -value = 0.00044347) and `sts_serial 3` (p -value = 0.99644404). These values still lie within the extreme tails of the random p -value distribution and do not directly imply structural weaknesses.

Furthermore, the generator passed all 160 tests in BigCrush without failure, confirming the absence of detectable bias across a comprehensive range of statistical probes, including spacing tests, autocorrelation, spectral dependencies, and Hamming weight transitions.

And also, the generator was subjected to PractRand stream-based testing up to 128 GB, exhibiting no failure or weak results. This places the generator's output quality within the statistical performance class of high-end PRNGs used in scientific computing and AI training workflows.

This behavior aligns with the theoretical expectation of non-invertible random mappings, where each iteration acts as a many-to-one function $F : S \rightarrow S$ that diffuses entropy uniformly over the state space. The uniform p -value distribution across all test suites thus provides empirical evidence of the mapping's chaotic diffusion properties, consistent with the random mapping theorem [9] [10] and the predicted cycle length $\lambda \approx 0.7824\sqrt{n}$.

Although the entropy quality of this algorithm is good and it passes all tests, this does not directly mean that the algorithm is secure for cryptography. The algorithm is designed with a new structure and has not yet been tested, with no specific evaluations against various types of attacks, so it still falls into the category of general-purpose.

2) *Avalanche-Effect Analysis with Multi-Seed Perturbation Models:* To comprehensively evaluate the avalanche characteristics of `ra_prng2`, three complementary experiments were conducted under distinct seed perturbation models. These models assess how minor variations in the initial seed propagate through the generator's iterative process, an essential property of diffusion in well-designed pseudorandom systems [11], [12]. Each experiment begins with the base seed `0x00000001` and generates 255 sequential 32-bit outputs. At every iteration, the bitwise difference between the modified-seed output and the baseline output is measured using the Hamming distance [13], representing the number of differing bits between corresponding outputs.

Three distinct perturbation schemes were employed:

- 1) **Bit-Flipping Avalanche Test.** Each modified seed is derived by flipping a single bit in the base seed:

$$\text{seed}_i = \text{seed}_0 \oplus (1 \ll i), \quad 0 \leq i < 32$$

This configuration measures orthogonal sensitivity across bit positions, indicating how individual input bits influence overall diffusion. In classical avalanche

analysis, strong diffusion is indicated when single-bit perturbations produce a 50% difference in output bits in under 3 iterations [14].

- 2) **Incremental-Seed Sensitivity Test.** The initial seed increases linearly as:

$$\text{seed}_i = \text{seed}_0 + (i + 1), \quad 0 \leq i < 32$$

This test evaluates continuity resistance, determining whether nearby seeds yield decorrelated outputs. High sensitivity under incremental variation indicates the generator’s resistance to local correlation traps, common weaknesses in low-entropy PRNG designs.

- 3) **Power-of-Two Seed Test.** Each seed is initialized as a single active bit at distinct positions:

$$\text{seed}_i = 1 \ll (i + 1), \quad 0 \leq i < 32$$

This isolates the contribution of each bit position as an independent entropy vector, revealing uniformity in how each seed bit contributes to the evolving state.

For each of the three experiments above, the per-iteration average of the Hamming distance across all 32 perturbed seeds was computed. The resulting comparison is shown in Fig. 2, where the horizontal axis represents the internal iteration index, and the vertical axis shows the average number of differing bits (0–32) between baseline and perturbed outputs. Each line represents one perturbation model: Bit-Flipping (blue), Incremental (orange), and Power-of-Two (green).

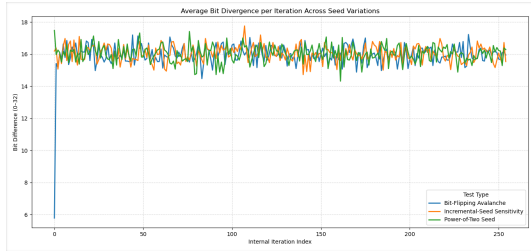


Fig. 2. Average bit divergence per iteration across three seed perturbation models: Bit-Flipping, Incremental, and Power-of-Two. The ideal diffusion reference (50% bit difference $\approx 16/32$ bits) is achieved after the first few iterations.

As illustrated in Fig. 2, all three perturbation models rapidly converge toward a stable diffusion level near 16 out of 32 bits (approximately 50%), consistent with the expected behavior of a high-entropy generator. The Bit-Flipping test reveals a minor delay for upper-bit flips (bit 5–32) within the first iteration, after which diffusion becomes uniform, suggesting transient local dependency in early state transitions. Conversely, both the Incremental and Power-of-Two seed tests show immediate and uniform diffusion from the first iteration, confirming that even simple arithmetic or isolated-bit seed perturbations lead to strongly decorrelated outputs. Together, these results confirm that `ra_prng2` achieves robust avalanche behavior with rapid entropy propagation and no detectable bias across different seed perturbation domains. Full source code for these tests is available in the `benchmark` folder on GitHub [15].

Beyond validating diffusion quality, these sensitivity results carry a structural implication. The immediate and sustained divergence observed across all three perturbation models — including the incremental test where seeds differ by as little as one unit — indicates that `ra_prng2`’s output stream is effectively unique per seed. Combined with the architecture’s deterministic reproducibility, this property constitutes a functional basis for lightweight data fingerprinting: a given seed deterministically produces a unique transformation trajectory that cannot be reproduced without knowledge of the exact initial condition. This is distinct from cryptographic commitment schemes; no security claim is made here. Rather, we identify this as an architectural property with potential utility in provenance-tracking and reproducible transformation pipelines, which we discuss further in Section IV.

B. Performance and Throughput

`ra_prng2` achieved a throughput of up to 8.16 Gbit in a single iteration completed within 1.073 seconds, equivalent to 950.6 Megabytes/s. In microbenchmarks, the algorithm reached an IPC (Instructions per cycle) of 3.72, which is relatively high, with minimal branch prediction. In shuffling applications, `ra_prng2` outperformed `chacha20` by up to $1,62\times$ faster in no-buffer call mode. This is due to the denser instructions of `ra_prng2`; although the operational cost is higher, the absence of I/O overhead keeps the generation process efficient. IPC was measured using `perf stat` under the same experimental conditions described in Section III-D.

C. Periodicity Analysis

A central property of any pseudo-random number generator (PRNG) is its *periodicity*, the number of iterations before the generator returns to a previously visited internal state. The theoretical period can be estimated from the number of reachable PRNG states, which generally depends on the PRNG’s ability to traverse every state without omission. In practice, perfect mixing is rare, and actual periods may be limited by local loops, unreachable states, or noninvertible processes [16], [17]. The following outlines the rough upper-bound estimation of the period before returning to the initial state. Formally, let $\mathcal{S}$ denote the finite state space of the generator, and let $F : \mathcal{S} \rightarrow \mathcal{S}$ represent the deterministic update operator defining one iteration of the generator. For discrete time steps $t \in \mathbb{N}$, the internal state evolves as

$$S_{t+1} = F(S_t),$$

with the orbit of an initial state S_0 defined as

$$\mathcal{O}(S_0) = \{S_0, F(S_0), F^2(S_0), \dots\}.$$

Since $\mathcal{S}$ is finite, every trajectory is eventually periodic: there exist integers $\mu \geq 0$ and $\lambda \geq 1$ such that

$$F^{\mu+\lambda}(S_0) = F^\mu(S_0),$$

where λ is called the *period length*. The goal of this section is to characterize λ for `ra_prng2`.

a) *Non-invertible structure.*: Unlike classical generators such as Mersenne Twister or xoshiro, which are designed as bijective or semi-bijective linear recurrences over $\mathbb{F}_2$, the core update operator of `ra_prng2` is *non-invertible*. This non-invertibility arises from the internal step that generates the constant term `cons` through a hash-like reduction:

$$\text{cons}_{t+1} = H(M_t),$$

where H combines multiple words of M_t using nonlinear XOR-aggregation. Because many distinct input states can yield the same 32-bit output, H acts as a many-to-one mapping, compressing information and breaking strict reversibility. Consequently, multiple distinct states can converge to the same future orbit, reducing the maximal achievable period compared to a perfectly invertible mapping.

b) *Contributions to state size.*: To estimate upper bounds on possible cycle lengths we separate the state into two classes of contributions: (i) components whose bits are *fully mutable* under the update (i.e., each bit may change through XOR/add/rotate operations), and (ii) components that are *permutative only* (their values are rearranged but not bitwise mutated).

In our `ra_prng2` instantiation the fully mutable components are the array `M` (256 words of 32 bits each), the per-iteration constant `cons` (32 bits), and the iteration counter `it` (32 bits). Hence the mutable bit count is

$$B_{\text{mut}} = 256 \cdot 32 + 32 + 32 = 8256.$$

The array `L` contributes only by permutations of its 256 entries; therefore its combinatorial contribution is $256!$, which satisfies (using Stirling's approximation)

$$\log_2(256!) \approx 1684.05.$$

c) *Raw upper bound.*: Multiplying the two contributions yields a raw upper bound on the number of distinct internal states:

$$|\mathcal{S}| \approx 2^{8256} \times 256! \approx 2^{8256} \times 2^{1684.05} \approx 2^{9940}.$$

We stress that this is a structural upper bound on the number of possible states (assuming full variability of the mutable components) and *not* a proven period.

d) *Heuristic theoretical periodicity for non-invertible mappings.*: Because `ra_prng2` includes a hash-like reduction when generating `cons` (a many-to-one step that compresses multiple words into a 32-bit value), the overall update map behaves non-invertibly. This non-invertibility implies that the mapping $F : \mathcal{S}_t \rightarrow \mathcal{S}_{t+1}$ cannot be represented as a bijection, since multiple distinct internal states can converge to the same successor.

Therefore, its long-term dynamics are more accurately modeled as a random many-to-one mapping rather than a deterministic permutation. Under this assumption, the random mapping theory provides an appropriate heuristic framework for estimating the expected cycle length and the statistical

behavior of orbit convergence. [9] [10] We approximate the theoretical periodicity using this heuristic,

$$\lambda \approx 0.7824 \sqrt{|\mathcal{S}|} \approx 0.7824 \cdot 2^{9940/2} = 0.7824 \cdot 2^{4970}.$$

Equivalently,

$$\lambda \approx 2^{4970 + \log_2(0.7824)} \approx 2^{4969.65},$$

which should be interpreted as a statistical/heuristic estimate rather than a rigorous period proof.

e) *Comparison to MT19937 and ra_prng3.*: For reference, the Mersenne Twister MT19937 has period $2^{19937} - 1$ [18]. Under the calculations above the raw upper bound exponent of `ra_prng2` (≈ 9940) and the heuristic period exponent (≈ 4969.65) are both *smaller* than the MT19937 exponent (19937). Thus MT19937's period remains substantially larger than the heuristic theoretical periodicity for `ra_prng2`.

If the design is extended to a 64-bit word size and all state words become fully mutable (the `ra_prng3` variant), then the total bit count becomes

$$B_3 = 2 \cdot 256 \cdot 64 + 64 + 64 = 32896,$$

giving a raw upper bound 2^{32896} and a corresponding heuristic many-to-one theoretical periodicity

$$\lambda_3 \approx 0.7824 \cdot 2^{32896/2} = 0.7824 \cdot 2^{16448} \approx 2^{16447.65}.$$

Note that while the raw bound (2^{32896}) exceeds the MT19937 exponent, the random-mapping heuristic (which takes the square root) yields an exponent (≈ 16447.65) that remains below 19937.

f) *Caveat.*: All expressions above are estimations: the raw bounds count combinatorial possibilities, while the $\sqrt{n}$ heuristic applies when the effective mapping on reachable states behaves like a random many-to-one map. A rigorous proof of exact period for the full-size generator is computationally infeasible.

D. Comparison with Other Algorithms

This section compares the statistics of `ra_prng2` with several other algorithms in terms of entropy quality, periodicity, RNG speed, shuffling efficiency, and internal architectural complexity. For a fair and standardized benchmark, all experiments were run in the same environment.

• Experimental Environment

All experiments were conducted on a personal computer with the following specifications:

- 1) Processor: Intel Core i3-1115G4 (2 cores, 4 threads, up to 4.10 GHz)
- 2) RAM: 8 GB DDR4
- 3) Operating System: Arch Linux (kernel 6.8.7-arch1-1)
- 4) Device: Acer Aspire 5 (model A514-54)

The algorithm implementations were written in C and compiled with `gcc` version 15.1.1 using the `-O3` flag. Statistical tests were performed with NIST STS version 2.1.2 and Dieharder version 3.31.1.

- **Random-Number Generation Speed**

The table below shows throughput in MB/s. The `ra_prng3` algorithm is a development version of this stable release but has not yet been further analyzed for other aspects. Measurements were taken by timing how fast the PRNG can generate 1 GiB of output.

TABLE I
PRNG SPEED BENCHMARK

Algorithm	Throughput (MB/s)
xoshiro256	3597.1
PCG32	3196.7
ra_prng3 (dev)	1539.6
Philox4x32	1339.8
ra_prng2	728.0
ChaCha20	564.8

1) *Shuffling Performance Evaluation:* The RNG implementations for shuffling are available in the GitHub repository. Benchmarks were performed by shuffling a dataset of 1,000,000 token items using the Fisher-Yates algorithm. Each experiment was executed 30 times and measured using `perf stat -r 30`. Two experimental settings were used: (1) *baseline per-call*, where each random index is generated by invoking the RNG function once without buffering, and (2) *buffered*, where PRNGs amortize random number generation by using a small block of precomputed random values.

The `ra_prng` family was evaluated only in the per-call setting, since its design intrinsically integrates shuffling within its state transition, making additional buffering redundant. In each iteration, `ra_prng2` performs internal swaps across the array state, rendering buffering irrelevant for this application.

Results for both configurations are shown in Table II. In the per-call configuration, `ra_prng2` and its successor `ra_prng3` require approximately twice the time to shuffle compared to the fastest algorithms, with an average shuffle time of around 0.12 seconds. Nevertheless, this performance gap is considerably smaller than in raw throughput benchmarks; for instance, `ra_prng2` is about 4.94× slower than `xoshiro256` in random number generation throughput, but only 1.99× slower in practical shuffling.

When buffering is introduced, most conventional PRNGs show only marginal improvements, typically less than 0.1 ms. However, `ChaCha20` exhibits a notable speedup: its shuffle time decreases from 0.196 seconds to 0.064 seconds, nearly a threefold improvement. As expected, the `ra_prng` family was not included in this comparison due to its integrated design, which inherently performs array mutation during each state transition rather than relying on separate random calls.

- **Entropy-Test Results** Tests were performed with NIST STS and Dieharder. The following table summarizes whether each algorithm passed all subtests in the respective tools.

TABLE II
SHUFFLE TIME FOR 1,000,000 ITEMS UNDER BOTH CONFIGURATIONS (PER-CALL AND BUFFERED), “DEV” REFERS TO STANDARD DEVIATION ACROSS 30 RUNS.

Algorithm	Per-call (No Buffering)			Buffered (16-word)		
	Times	Dev	IPC	Times	Dev	IPC
xoshiro256	0.06101	2.81%	1.60	0.06085	3.51%	1.63
PCG32	0.05992	1.60%	1.59	0.05931	2.06%	1.64
ra_prng3	0.12159	1.36%	3.47	-	-	-
Philox4x32	0.06622	3.75%	1.43	0.06249	4.43%	1.53
ra_prng2	0.12159	1.36%	3.46	-	-	-
ChaCha20	0.19645	2.09%	1.95	0.06452	2.73%	1.64

TABLE III
DIEHARDER TEST SUMMARY (114 SUBTESTS) AND ESTIMATED THEORETICAL PERIODICITY

Algorithm	PASSED	WEAK	FAILED	Theoretical Period
xoshiro256	111	3	0	2^{256}
PCG32	112	2	0	2^{64}
ra_prng3	-	-	-	$2^{16447.65}$
Philox4x32	112	1	1	2^{128}
ra_prng2	112	2	0	$2^{4969.65}$
ChaCha20	110	4	0	2^{512}

a) *Notes on the WEAK Flag in Dieharder:* Some algorithms show a WEAK flag in a small number of Dieharder subtests. It is important to note that WEAK does not automatically imply a critical weakness or failure in randomness quality; it typically appears when a p-value is near the extreme tail (too high or too low), which can occur even for statistically strong random data generators. A fair evaluation should consider the overall distribution of results and the number of subtests passed.

- **xoshiro256:** WEAK on `diehard_opso` and two variants of `rgb_lagged_sum` (p-values 0.9966 and 0.0013).
- **PCG32:** WEAK on two `rgb_lagged_sum` subtests (p-values ≈ 0.998).
- **ra_prng2:** WEAK on `sts_serial 3` and `rgb_lagged_sum 24` (extreme p-values).
- **Philox4x32:** FAILED on `rgb_lagged_sum 31` ($p = 9e-8$) and WEAK on one `marsaglia_tsang_gcd` subtest.
- **ChaCha20:** WEAK on `parking_lot`, `3dsphere`, and two `rgb_lagged_sum` subtests (e.g., $p = 0.00029$ and 0.9986).

IV. DISCUSSION AND FUTURE WORK

An interesting observation from the benchmarking results is that although `ra_prng3` achieves nearly twice the raw throughput of `ra_prng2`, both algorithms produce nearly identical shuffling performance. This gap is attributable to significant bit waste in Lemire’s fast reduction step, where a large portion of the generated bits cannot be used for indexing. This suggests a concrete direction for future optimization: redesigning output usage to minimize discarded bits, which would allow throughput gains to translate directly into shuffling gains.

The buffering results further illustrate this dynamic. Most conventional PRNGs show only marginal improvement under buffering, while ChaCha20 exhibits a nearly threefold speedup — from 0.196s to 0.064s — by amortizing generation cost across precomputed blocks. The `ra_prng` family was excluded from this comparison by design: because state mutation and output extraction occur within the same loop, buffering is structurally redundant. This integration is a deliberate architectural choice, not a limitation.

The consistently high IPC (≈ 3.5) with low deviation ($\approx 1.3\%$) across both `ra_prng2` and `ra_prng3` warrants separate attention. This indicates that the instruction stream is not only dense but stable — the branchless design eliminates pipeline stalls and prediction errors that typically inflate deviation in branch-heavy generators. While high IPC does not directly translate to faster wall-clock shuffling in the current implementation, it suggests that the architecture saturates available execution units efficiently. Future work targeting lighter bitwise operations — while preserving the branchless structure — could convert this execution density into measurable throughput improvements.

Beyond performance, the avalanche analysis presented in Section III reveals a property with implications that extend past conventional PRNG evaluation. The immediate and sustained output divergence observed under all three perturbation models — particularly under incremental seeding where initial conditions differ by a single unit — indicates that each seed generates a structurally unique transformation trajectory. Combined with the architecture’s strict determinism, this constitutes a functional basis for lightweight data fingerprinting: given a fixed seed, `ra_prng2` produces a reproducible permutation sequence that is practically irrecoverable without knowledge of the exact initial condition, due to the non-invertibility of the hash-like reduction at each reseeding step. This is architecturally distinct from cryptographic schemes — no resistance to adversarial attack is claimed. Rather, we identify this as a system-level primitive suitable for provenance tracking, reproducibility verification, and data transformation auditing in non-adversarial environments. Formalizing this application and evaluating its practical utility constitutes a concrete direction for subsequent work.

V. CONCLUSION

This paper introduces `ra_prng2`, an array-native PRNG architecture that inverts the conventional relationship between randomness and permutation: rather than using random values to drive shuffling, the architecture treats permutation of its internal array state as the generative mechanism itself. This structural inversion eliminates the call boundary between generator and shuffler, enabling both processes to proceed within a single branchless execution loop without external function calls or reseeding overhead.

Benchmarking against established PRNGs — `xoshiro256`, `PCG32`, `Philox4x32`, and `chacha20` — demonstrates that `ra_prng2` achieves competitive shuffling performance in the per-call configuration, with the smallest

deviation across runs (1.36%) and the highest IPC among all evaluated algorithms (≈ 3.46). Statistical evaluation confirms near-ideal output quality: `ra_prng2` passes all NIST STS, Dieharder, PractRand, and BigCrush test suites, with entropy reaching 7.99999 bits/byte and an avalanche sensitivity of 49.9%. The estimated theoretical period of $2^{4969.65}$ is extended to $2^{16447.65}$ in `ra_prng3`, without sacrificing statistical quality, and with nearly double the raw throughput.

The avalanche analysis further reveals that `ra_prng2` exhibits strong deterministic sensitivity to initial conditions: minor seed perturbations produce immediate and sustained output divergence across three independent perturbation models. This property, combined with the architecture’s non-invertible reseeding step, positions `ra_prng2` as a candidate primitive for lightweight data fingerprinting and provenance tracking in non-adversarial transformation pipelines.

Future work will pursue three directions: reducing bit waste in Lemire’s fast reduction to close the gap between raw throughput and shuffling performance; further developing `ra_prng3` as a 64-bit extension with full state mutability; and formalizing the fingerprinting application through empirical evaluation of seed-to-trajectory uniqueness and its utility in reproducible data transformation auditing.

REFERENCES

- [1] R. Durstenfeld, “Algorithm 235: Random permutation,” *Communications of the ACM*, vol. 7, no. 7, p. 420, 1964.
- [2] M. Penschuck *et al.*, “Engineering shared-memory parallel shuffling to generate random ...,” *arXiv preprint arXiv:2302.03317*, 2023.
- [3] D. Lemire, “Fast random integer generation in an interval,” *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, vol. 29, no. 1, pp. 3:1–3:12, 2019.
- [4] hamzy hams, “`ra_prng2`: Novel pseudorandom number generator structure with internal state mutation,” https://github.com/hamzy-hams/ra_prng/tree/main/ra_prng2/src, 2025, accessed: 2025-06-16.
- [5] J. Walker, “Ent: A pseudorandom number sequence test program,” <https://www.fourmilab.ch/random/>, 2008, accessed: 2025-06-16.
- [6] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [7] A. Rukhin *et al.*, “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” National Institute of Standards and Technology (NIST), Tech. Rep. Special Publication 800-22 Revision 1a, 2001. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final>
- [8] R. G. Brown, “Dieharder: A random number test suite,” <https://webhome.phy.duke.edu/~rgb/General/dieharder.php>, 2004, accessed: 2025-06-16.
- [9] S. R. Finch, “Components and cycles of random mappings,” 2022, arXiv preprint, submitted 11 May 2022. [Online]. Available: <https://arxiv.org/abs/2205.05579>
- [10] L. Mutafchiev and S. R. Finch, “On the deepest cycle of a random mapping,” *Journal of Combinatorial Theory, Series A*, vol. 206, p. 105875, 2024.
- [11] A. F. Webster and S. E. Tavares, “On the design of s-boxes,” *Advances in Cryptology — CRYPTO ’85*, pp. 523–534, 1986.
- [12] J. Daemen and V. Rijmen, *The Design of Rijndael: AES—The Advanced Encryption Standard*. Springer, 2002.
- [13] R. W. Hamming, “Error detecting and error correcting codes,” *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.
- [14] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed. Chapman and Hall/CRC, 2005.
- [15] hamzy hams, “`ra_prng2`: Novel pseudorandom number generator structure with internal state mutation,” https://github.com/hamzy-hams/ra_prng/blob/main/ra_prng2/benchmark/avalanche_effect_analysis.py, 2025, accessed: 2025-09-25.

- [16] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [17] P. L'Ecuyer, "A note on the period of combined multiple recursive random number generators," *Statistics & Probability Letters*, vol. 9, no. 4, pp. 335–339, 1990.
- [18] M. Matsumoto and T. Nishimura, "Mersenne twister: a 623-dimensionally equidistributed uniform pseudorandom number generator," pp. 3–30, 1998.